



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY



分布式机器学习(III)

本地更新随机梯度下降

王宏志

哈尔滨工业大学

计算学部海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 本地更新随机梯度下降
- 2 自适应通信方案
- 3 弹性平均SGD
- 4 重叠本地SGD

目录

Contents

- 1 本地更新随机梯度下降
- 2 自适应通信方案
- 3 弹性平均SGD
- 4 重叠本地SGD

回顾：同步SGD算法

在每次迭代中

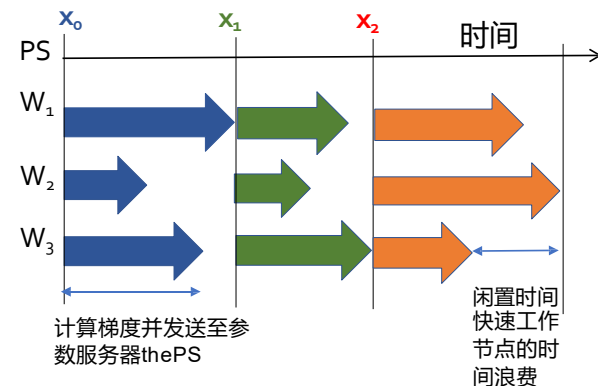
1. 将当前参数向量 $\mathbf{x}_t$ 发送至所有 m 个工作节点
2. 每个工作节点 i 计算梯度

使用从数据集分区 D_i 中均匀有放回抽取的 b 个样本计算小批量梯度： $g(\mathbf{x}_t; \xi_i) = \frac{1}{b} \sum_{j=1}^b \nabla f(\mathbf{x}; \xi_{i,j})$

3. 参数服务器收集这 m 个小批量梯度并按如下方式更新参数向量：

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{1}{m} \sum_{i=1}^m g(\mathbf{x}_t; \xi_i)$$

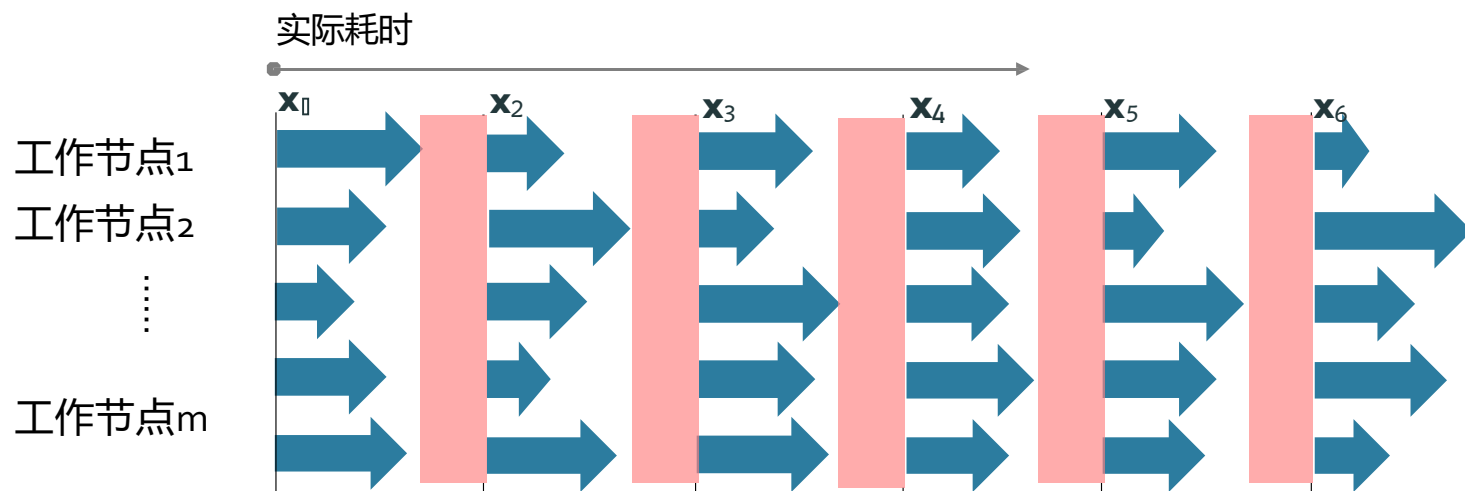
在此图中，未考虑梯度通信的延迟



每次迭代的通信延迟时间

粉色标记的通信延迟区块包含以下内容

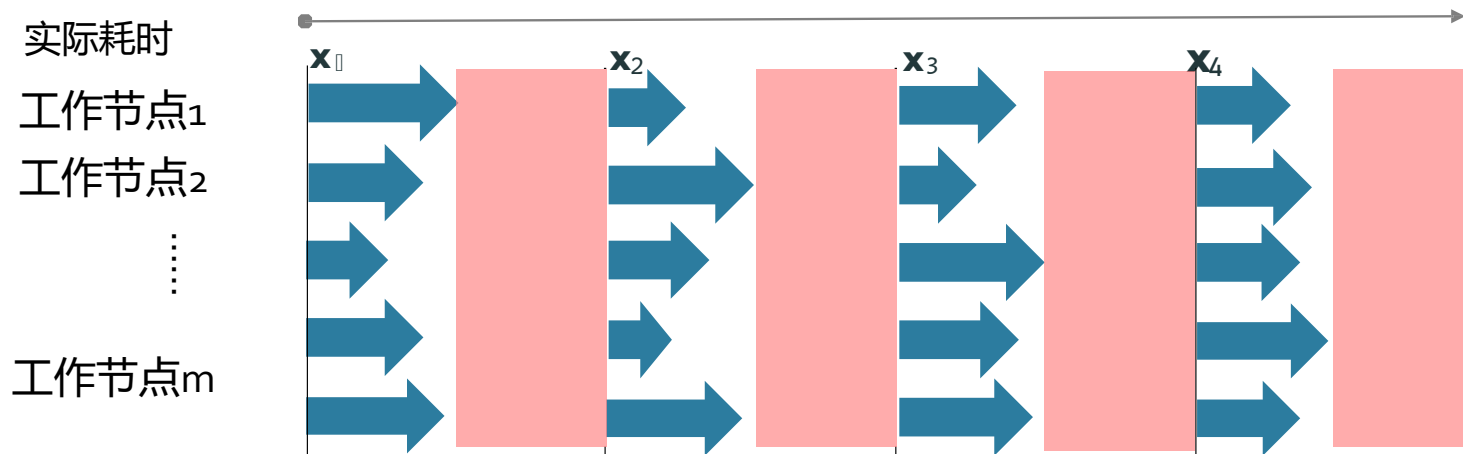
- 接收来自工作节点的梯度 $g(\mathbf{x}_t)$ 所需时间
- 将 $\mathbf{x}_t$ 更新至 $\mathbf{x}_{t+1}$ 的时间（通常较短）
- 将更新后的 $\mathbf{x}_{t+1}$ 发送回工作节点的时间



每次迭代的通信延迟时间

粉色标记的通信延迟区块包含以下内容

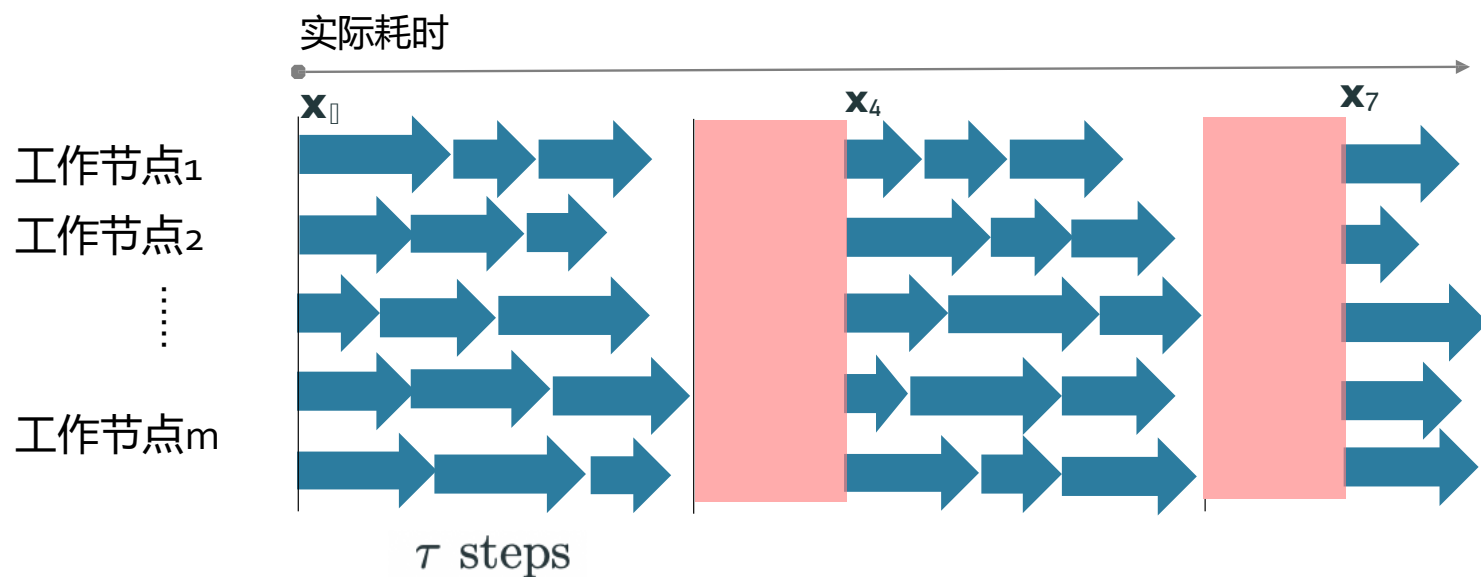
- 接收来自工作节点的梯度 $g(\mathbf{x}_t)$ 所需时间
- 将 $\mathbf{x}_t$ 更新至 $\mathbf{x}_{t+1}$ 的时间（通常较短）
- 发送更新后的 $\mathbf{x}_{t+1}$ 所需时间发送更新



通信延迟会随着模型大小的增加而上升，即向量 $\mathbf{x}$ 的规模

解决方案：执行多次本地更新

- 每个工作节点先进行 τ 次本地更新，再将更新后的模型发送至参数服务器进行聚合
- 通过降低通信频率，使通信延迟分摊到 τ 次迭代中
- 可在相同时间内完成更多SGD迭代

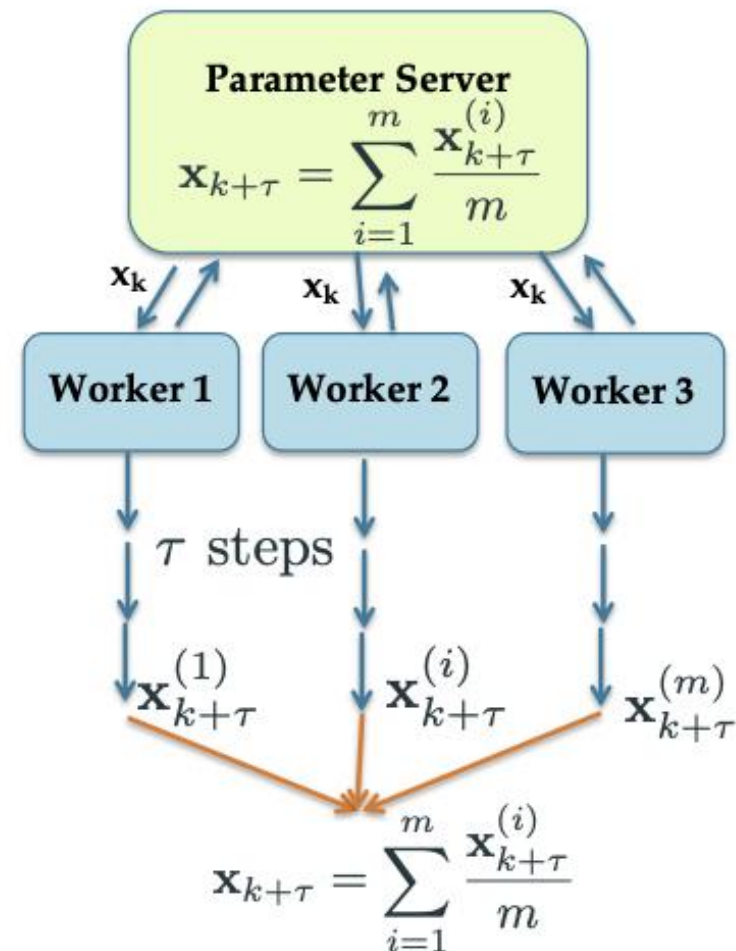


本地更新SGD算法

本地更新SGD算法以‘通信轮次’或‘训练轮次’运行。每轮执行以下操作：

- 工作节点 i 从参数服务器获取当前模型 $\mathbf{x}_k$
- 使用本地数据 D_i 进行 τ 次SGD更新
- 由于各节点数据 D_i 不同，导致最终模型 $\mathbf{x}_{k+\tau}^{(i)}$ 将存在差异
- 参数服务器将各节点模型 $\mathbf{x}_{k+\tau}^{(i)}$ 按以下方式聚合

$$\mathbf{x}_{k+\tau} = \sum_{i=1}^m \frac{\mathbf{x}_{k+\tau}^{(i)}}{m}$$



本地更新SGD算法

- 因此，第 i -个工作节点上本地随机梯度下降的有效更新规则由以下公式给出

$$\mathbf{x}_{k+1}^{(i)} = \begin{cases} \frac{1}{m} \sum_{j=1}^m [\mathbf{x}_k^{(j)} - \eta g(\mathbf{x}_k^{(j)}; \xi_k^{(j)})], & (k+1) \bmod \tau = 0 \\ \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)}; \xi_k^{(i)}), & \text{otherwise} \end{cases}$$

- 当通信周期 $\tau=1$ 时，这将退化为完全同步的随机梯度下降（SGD）更新规则

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \left[\frac{1}{m} \sum_{i=1}^m g(\mathbf{x}_k; \xi_k^{(i)}) \right].$$

其中 $\mathbf{x}_k^{(i)}$ 表示第 i -个工作节点在第 k 次迭代结束时的模型参数向量

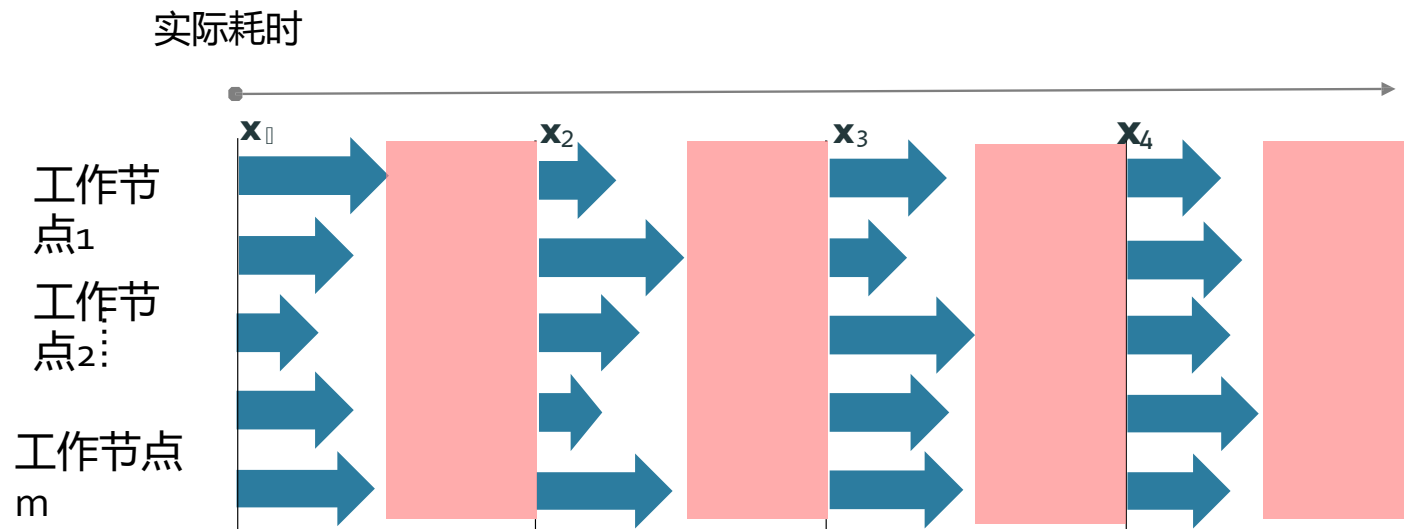
η 为学习率， m 是工作节点的数量

正交解决方案：梯度压缩

与降低通信频率不同

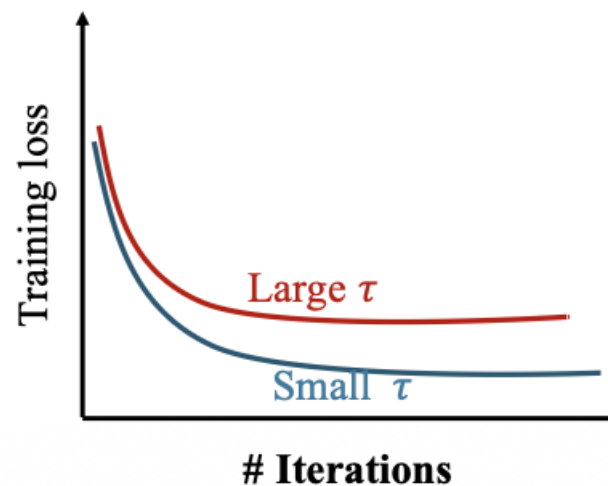
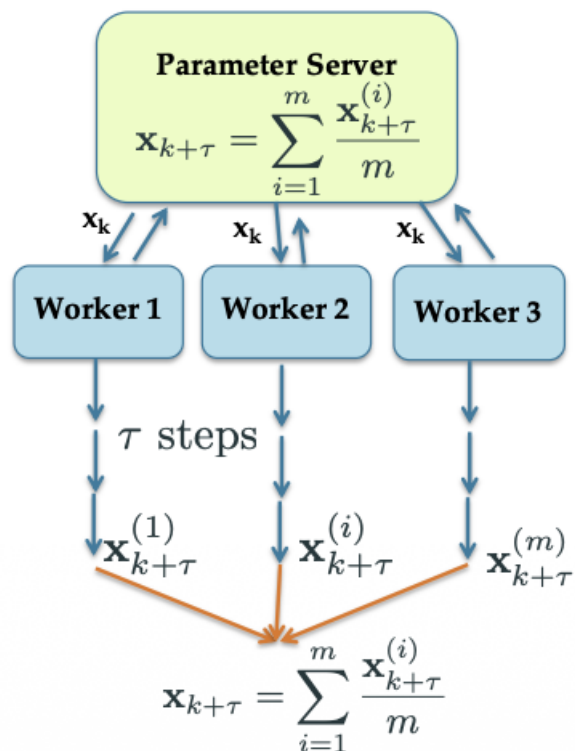
可通过压缩梯度 $g(x)$ 来减少每次迭代中传输的数据量

- 此方法可与本地更新结合，进一步减少通信开销
- 本地更新SGD：误差收敛性分析



τ 如何影响误差收敛?

- 更大的 τ 可能导致较差的错误收敛，因为节点处的局部值彼此发散。
- 将根据 τ 获得一个误差收敛界限。



本地更新SGD的收敛性

- **Lipschitz平滑性**: 目标函数 $F(\mathbf{x})$ 可微且满足 L -Lipschitz平滑条件,

$$\|\nabla F(\mathbf{x}) - \nabla F(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$$

- **下界 $F_{\inf}$** : 目标函数 $F(\mathbf{x})$ 有下界 $F_{\inf}$, 即,

$$F(\mathbf{x}) \geq F_{\inf} \text{ 对所有 } \mathbf{x}$$

- **无偏梯度**: 随机梯度 $g(\mathbf{x}; \xi)$ 是 $\nabla F(\mathbf{x})$ 的无偏估计, 即

$$\mathbb{E}_{\xi}[g(\mathbf{x}; \xi)] = \nabla F(\mathbf{x})$$

- **有界方差**: 随机梯度 $g(\mathbf{x}; \xi)$ 具有有界方差, 即

$$\text{Var}(g(\mathbf{x}; \xi)) \leq \sigma^2$$

$$\mathbb{E}_{\xi}[\|g(\mathbf{x}; \xi)\|^2] \leq \|\nabla F(\mathbf{x})\|^2 + \sigma^2$$

本地更新SGD的收敛界

本地更新SGD的收敛性

对于一个**L-光滑函数**，若 $\eta L + \eta^2 L^2 \tau(\tau - 1) \leq 1$ ，并且初始点为 $\mathbf{x}_1$ ，那么经过 t 次本地更新SGD迭代后， $F(\mathbf{x}_t)$ 满足如下界：

$$\mathbb{E} \left[\frac{1}{t} \sum_{k=1}^t \|\nabla F(\mathbf{x}_k)\|^2 \right] \leq \frac{2[F(\mathbf{x}_1) - F_{\inf}]}{\eta t} + \frac{\eta L \sigma^2}{m} + \eta^2 L^2 \sigma^2 (\tau - 1)$$

其中， $\mathbf{x}_k$ 表示第 k 次迭代时的**平均模型**。

- 上述分析未假设强凸性——因此左侧为梯度均值而非最优性差距
- 对于非凸函数只能展示收敛于 $F_{\inf}$ 目标函数的下界而非真正的最优值 F^* 。

本地更新SGD的收敛界

本地更新SGD的收敛性

对于一个**L-光滑函数**，若 $\eta L + \eta^2 L^2 \tau(\tau - 1) \leq 1$ ，并且初始点为 $\mathbf{x}_1$ ，那么经过 t 次本地更新SGD迭代后， $F(\mathbf{x}_t)$ 满足如下界：

$$\mathbb{E} \left[\frac{1}{t} \sum_{k=1}^t \|\nabla F(\mathbf{x}_k)\|^2 \right] \leq \frac{2[F(\mathbf{x}_1) - F_{\text{inf}}]}{\eta t} + \frac{\eta L \sigma^2}{m} + \eta^2 L^2 \sigma^2 (\tau - 1)$$

其中， $\mathbf{x}_k$ 表示第 k 次迭代时的**平均模型**。

- 当通信周期 $\tau=1$ ，该界限即为小批量SGD的收敛界限。
- 随着 τ 的增加，误差下界的最后一项变大，这验证了的直觉：误差下界会随着 τ 增加。

本地更新SGD的收敛界

本地更新SGD的收敛性

对于一个**L-光滑函数**，若 $\eta L + \eta^2 L^2 \tau(\tau - 1) \leq 1$ ，并且初始点为 $\mathbf{x}_1$ ，那么经过 t 次本地更新SGD迭代后， $F(\mathbf{x}_t)$ 满足如下界：

$$\mathbb{E} \left[\frac{1}{t} \sum_{k=1}^t \|\nabla F(\mathbf{x}_k)\|^2 \right] \leq \frac{2[F(\mathbf{x}_1) - F_{\inf}]}{\eta t} + \frac{\eta L \sigma^2}{m} + \eta^2 L^2 \sigma^2 (\tau - 1)$$

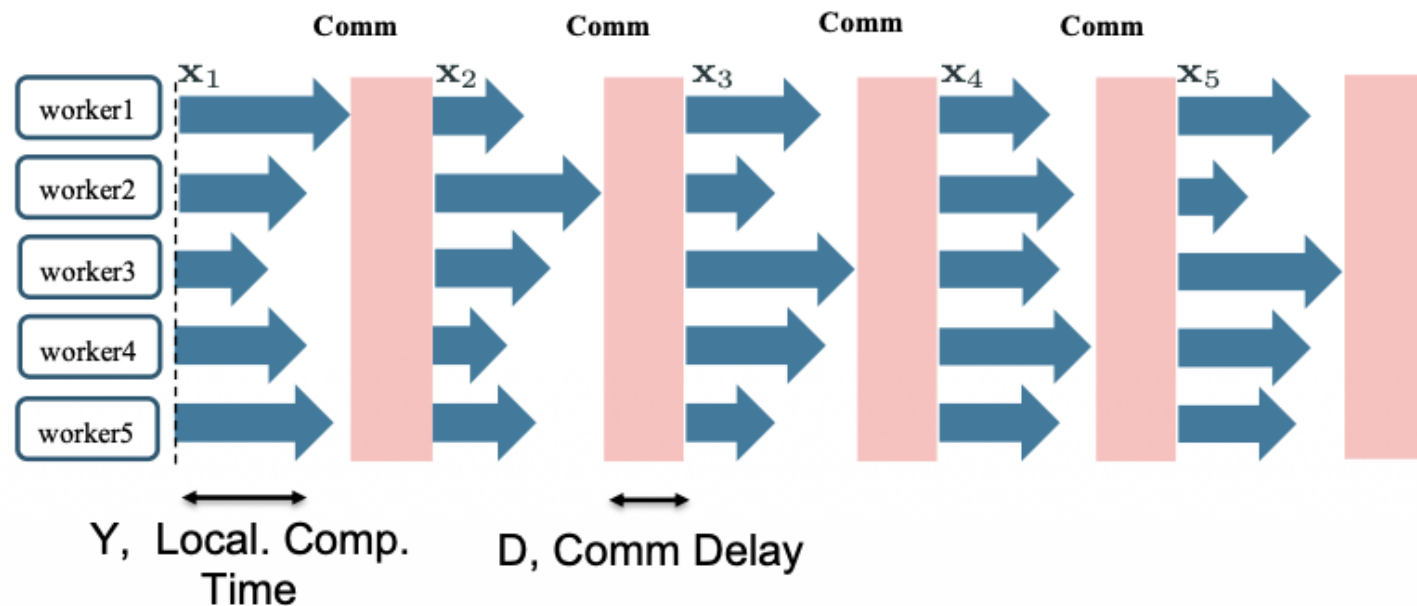
其中， $\mathbf{x}_k$ 表示第 k 次迭代时的**平均模型**。

其中 $\mathbf{x}_k$ 表示第 k 次迭代的平均模型

- 当通信周期 $\tau=1$ ，该界限即为小批量SGD的收敛界限。
- 随着 τ 的增加，误差下限中的最后一项会线性增长，这验证了的直觉：误差下限随 τ 的增加而上升

同步SGD的每次迭代时间

- 通信延迟通过随机变量 D 建模表示, 其中
- 执行一次本地更新的时间由随机变量 Y 建模表示
且假设其在工作节点和小批量数据间独立同分布 (i.i.d)
- 每次迭代的预期运行时间为: $\mathbb{E}[T_{sync}] = \mathbb{E}[Y_{m:m}] + \mathbb{E}[D]$

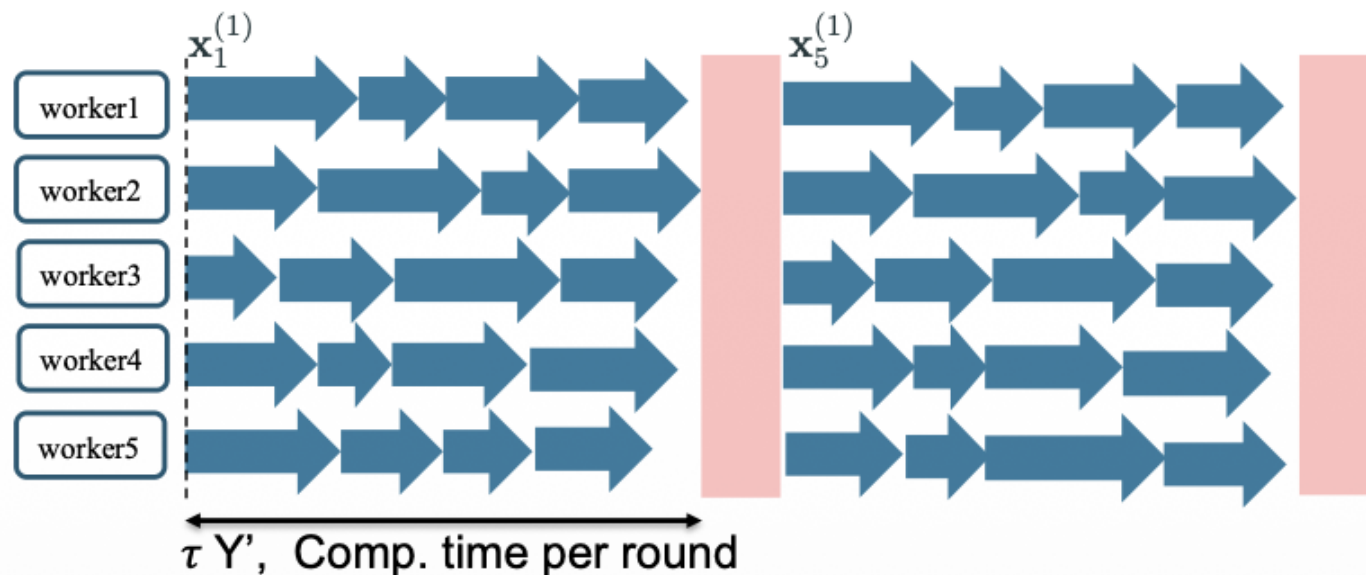


本地更新SGD的每次迭代时间

- 执行一次本地更新的时间由随机变量 Y 建模表示
且假设其在各工作节点和小批量数据间独立同分布 (i.i.d)
- 每轮总本地计算时间为 $T_{comp} = \tau Y' = (Y_1 + Y_2 + \dots + Y_\tau)$

$$Y' = \frac{(Y_1 + Y_2 + \dots + Y_\tau)}{\tau}, \text{ } Y \text{ 的独立同分布实现的平均值}$$

- 问题1: Y' 的期望值是多少?
- 问题2: Y' 的方差是多少?
- Y' 与 Y 具有相同均值但方差更低

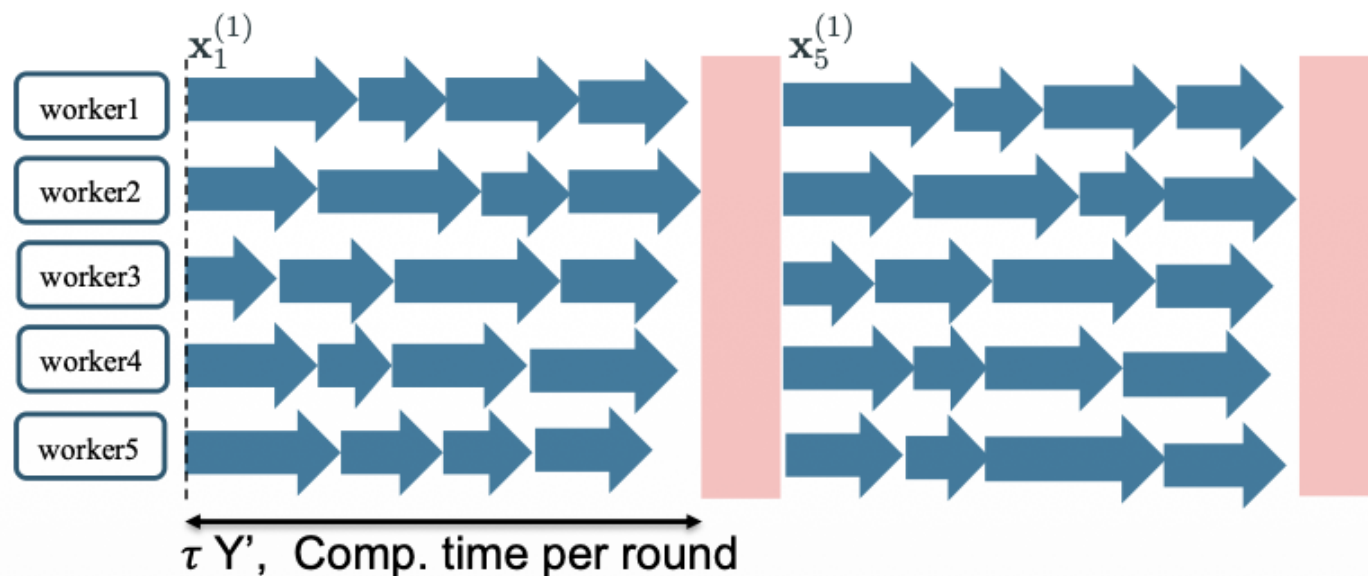


本地更新SGD的每次迭代时间

- 执行一次本地更新的时间由随机变量 Y 建模表示
且假设其在各工作节点和小批量数据间独立同分布 (i.i.d)
- 每轮总本地计算时间为 $T_{comp} = \tau Y' = (Y_1 + Y_2 + \dots + Y_\tau)$

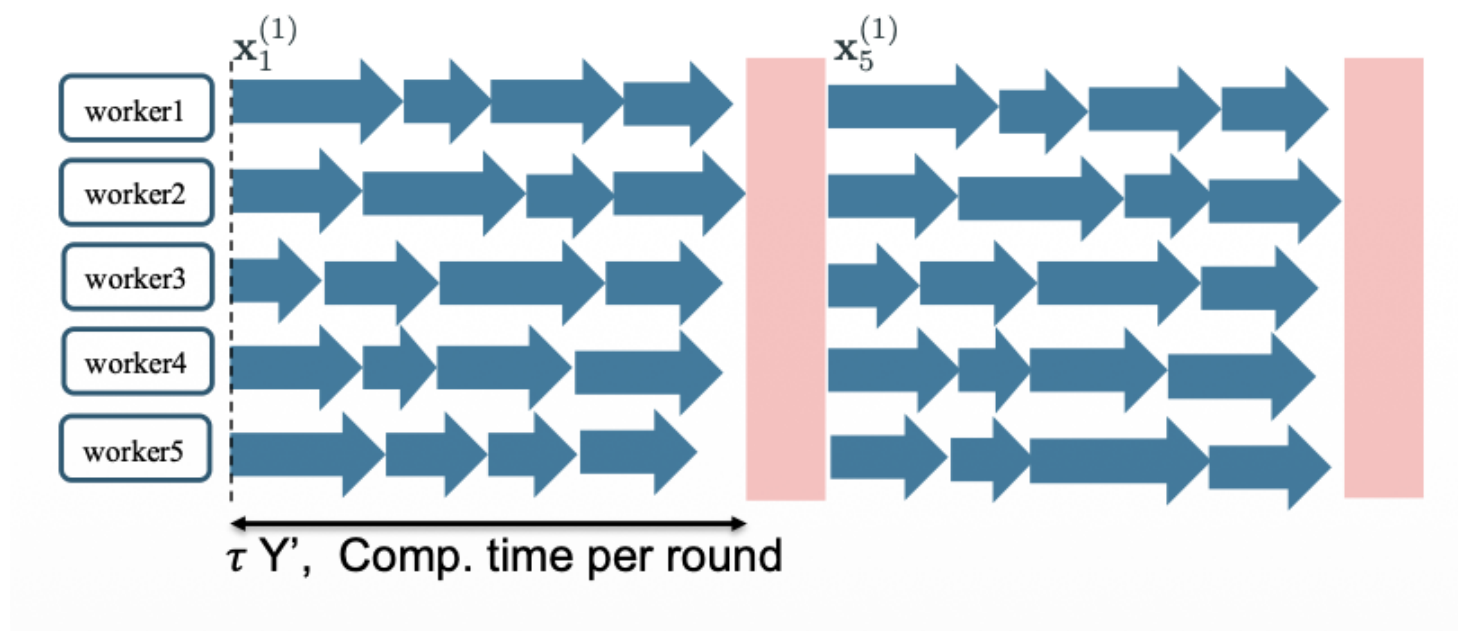
$$Y' = \frac{(Y_1 + Y_2 + \dots + Y_\tau)}{\tau}, \text{ } Y \text{ 的独立同分布实现的平均值}$$

- 问题1: Y' 的期望值是多少? $\mathbb{E}[Y'] = \mathbb{E}[Y]$
- 问题2: Y' 的方差是多少? $\text{Var}[Y'] = \text{Var}[Y]/\tau$
- Y' 与 Y 具有相同均值但方差更低



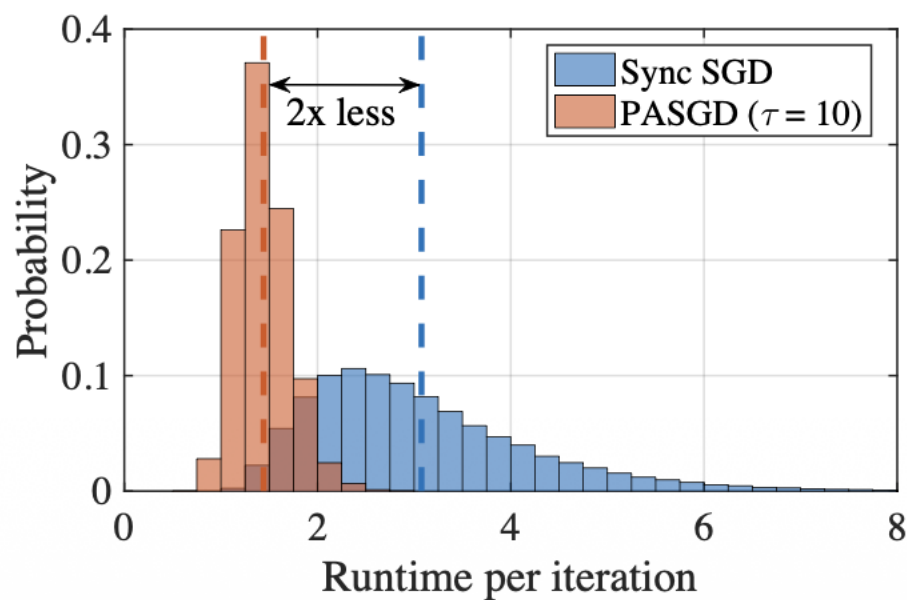
本地更新SGD的每次迭代时间

- 每次迭代的预期运行时为: $\mathbb{E}[T_{local}] = \mathbb{E}[Y'_{m:m}] + \frac{\mathbb{E}[D]}{\tau}$
- 让将此与同步SGD的运行时间进行比较 $\mathbb{E}[T_{sync}] = \mathbb{E}[Y_{m:m}] + \mathbb{E}[D]$
- 预期运行时间通过两种方式减少:
 - 通信延迟 $\mathbb{E}[D]$ 被除以 τ
 - $\mathbb{E}[Y'_{m:m}]$ 比 $\mathbb{E}[Y_{m:m}]$ 小, 因为 Y' 具有更低的方差——缓解了掉队者问题



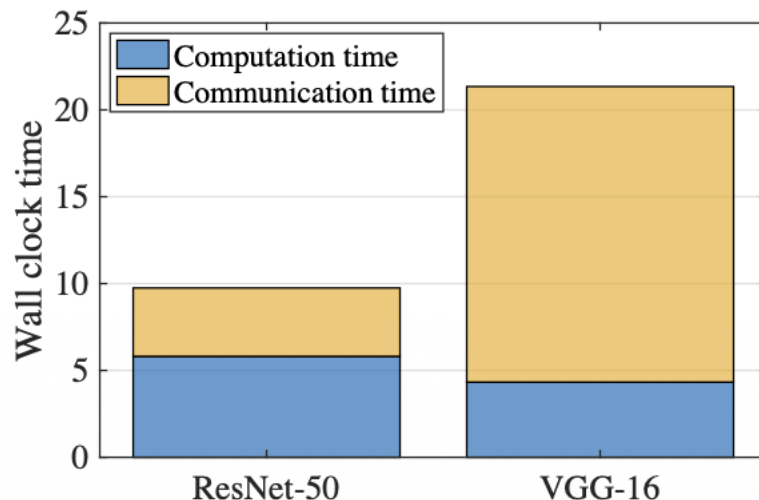
本地更新SGD的每次迭代时间

- 每次迭代的预期运行时为: $\mathbb{E}[T_{local}] = \mathbb{E}[Y'_{m:m}] + \frac{\mathbb{E}[D]}{\tau}$
- 本地SGD同时降低了运行时间的均值和方差



加速比相对于同步式SGD

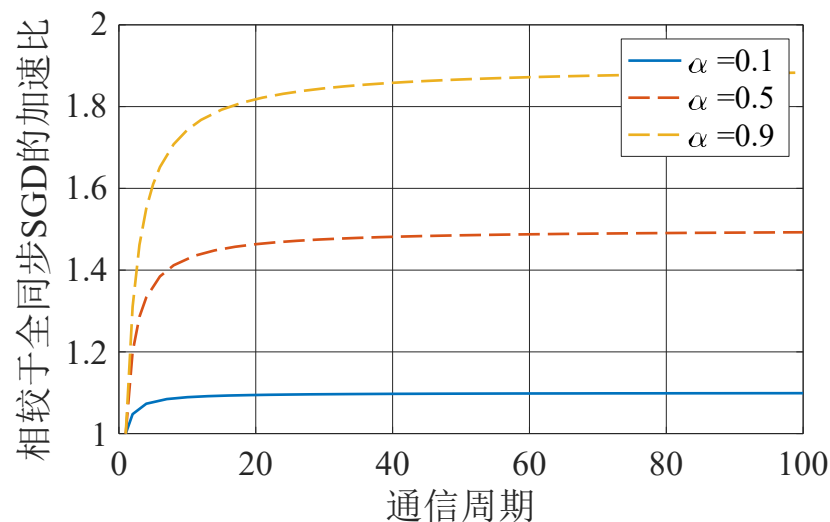
- 假设 Y 和 D 是常数,且它们的比例,即通信时间与计算时间之比为 $\alpha = D/Y$
- 参数 α 的取值取决于所训练模型的规模,可用通信带宽以及其他系统参数
- 如下文所示,VGG16的 α 值高于ResNet16,原因在于VGG16是一个更庞大的神经网络



加速比相对于同步式SGD

- 假设 Y 和 D 是常数,且它们的比例,即通信时间与计算时间之比为 $\alpha = D/Y$
- 本地更新SGD相对于同步式SGD的加速比由以下公式给出
- 当 α 增大时,本地更新SGD相比同步式SGD能带来更显著的加速效果

$$\frac{\mathbb{E}[T_{sync}]}{\mathbb{E}[T_{local}]} = \frac{Y + D}{Y + D/\tau} = \frac{1 + \alpha}{1 + \alpha/\tau}$$



结合误差与运行时分析

- 经过 t 次迭代后的误差被限定于

$$\mathbb{E} \left[\frac{1}{t} \sum_{k=1}^t \|\nabla F(\mathbf{x}_k)\|^2 \right] \leq \frac{2[F(\mathbf{x}_1) - F_{\inf}]}{\eta t} + \frac{\eta L \sigma^2}{m} + \eta^2 L^2 \sigma^2 (\tau - 1)$$

- 每次迭代的运行时（在常量 Y 和 D 下）为 $T_{local} = Y + D/\tau$
- 因此，在时间 T 时的误差被限定于

$$\text{Error at time } T \leq \frac{2[F(\mathbf{x}_1) - F_{\inf}]}{\eta T} \left(Y + \frac{D}{\tau} \right) + \frac{\eta L \sigma^2}{m} + \eta^2 L^2 \sigma^2 (\tau - 1)$$

结合误差与运行时分析

- 时间 T 时的误差被限定于

$$\frac{2[F(\mathbf{x}_1) - F_{\inf}]}{\eta T} \left(Y + \frac{D}{\tau} \right) + \frac{\eta L \sigma^2}{m} + \eta^2 L^2 \sigma^2 (\tau - 1)$$

- 随着 τ 增大, 收敛速度加快 (首项更小), 但误差下限 (末项) 会更高

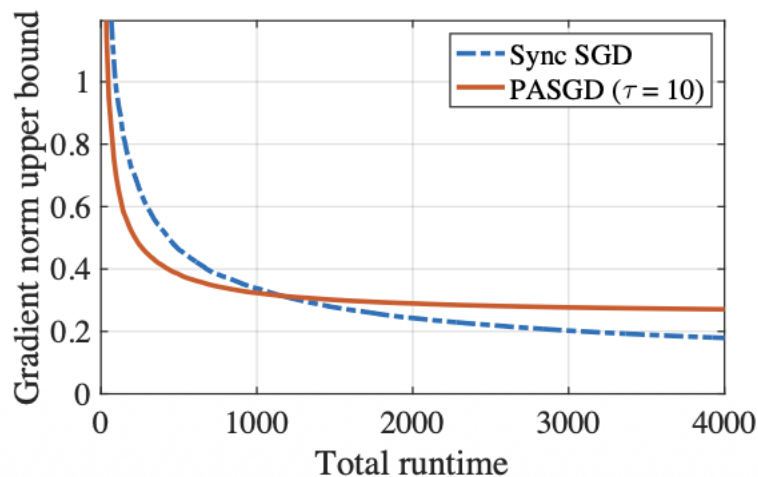
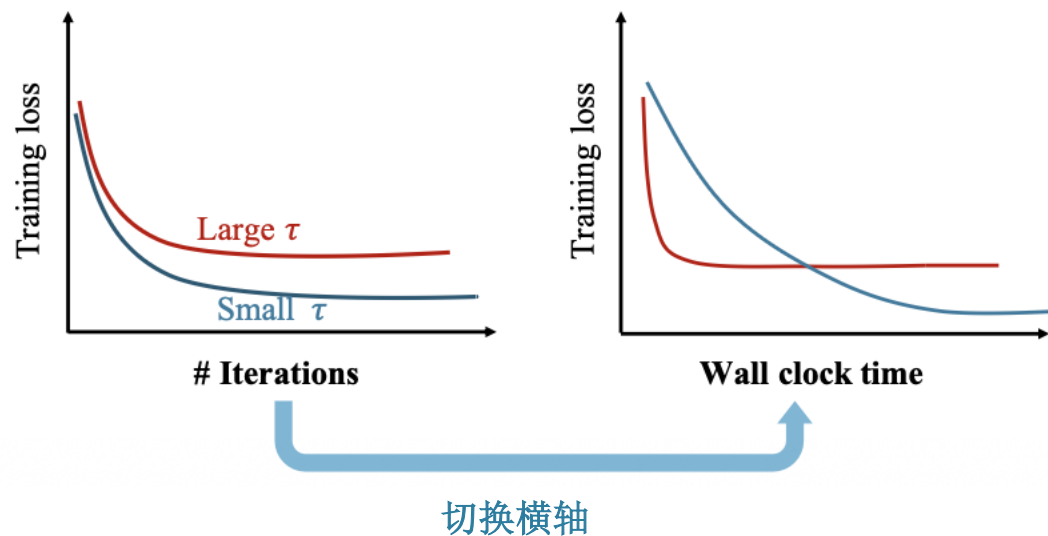


图1: 不同 τ 值的误差界对比

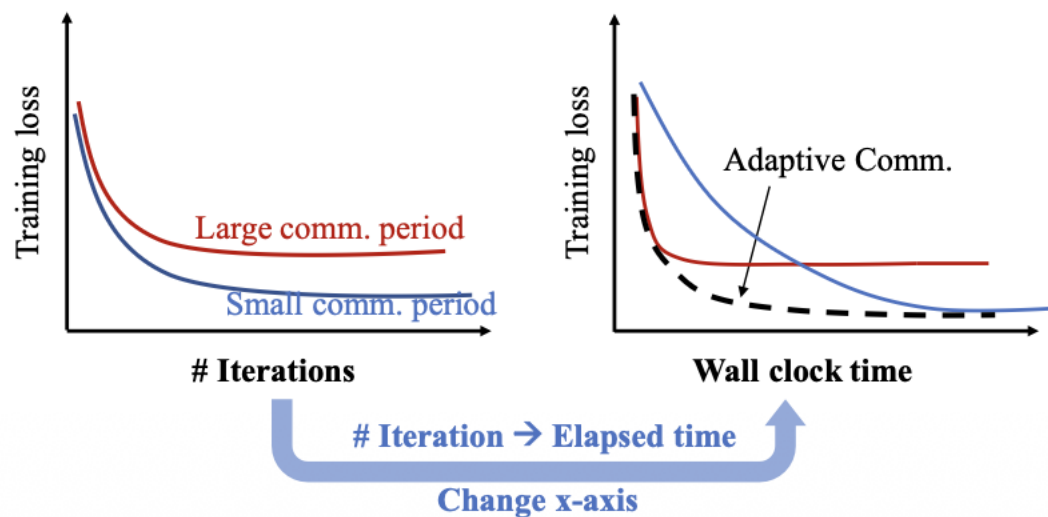
结合误差与运行时分析

- 较大的 τ 会导致更差的误差收敛（更高的误差下限）
- 较大的 τ 使每次迭代的运行时间更短
- 训练初期 τ 较大更有利，而在训练接近尾声时，较小的 τ 更优。



动态调整 τ 的动机

- 若从误差与迭代次数的权衡角度考虑，较大的 τ 会导致收敛性变差
- 但就误差与实时时间的权衡而言，初期采用较大 τ 可加速收敛，而在训练后期逐步减小 τ 有助于降低最终误差下限



想要制定一个调整 τ 的训练计划

τ 的启发式调度

- 思路：通过最小化误差上界，在每个实时时刻 T 找到最佳 τ

$$T \leq \frac{2[F(\mathbf{x}_1) - F_{\inf}]}{\eta T} \left(Y + \frac{D}{\tau} \right) + \frac{\eta L \sigma^2}{m} + \eta^2 L^2 \sigma^2 (\tau - 1)$$

- 将右侧表达式对 τ 求导并令其等于0，可得

$$-\frac{2[F(\mathbf{x}_1) - F_{\inf}]}{\eta T} \left(\frac{D}{\tau^2} \right) + \eta^2 L^2 \sigma^2 = 0$$
$$\tau^* = \sqrt{\frac{2[F(\mathbf{x}_1) - F_{\inf}] D}{\eta^3 L^2 \sigma^2 T}}$$

调整调度以适应实际约束

- 思路：通过最小化误差上界，在每个实时时刻 T 找到最佳 τ

$$\tau^* = \sqrt{\frac{2 [F(\mathbf{x}_1) - F_{\text{inf}}] D}{\eta^3 L^2 \sigma^2 T}}$$

- 能否直接应用于实践？遗憾的是，不能
- 目标函数及其随机梯度的属性参数 $(F(\mathbf{x}_1) - F_{\text{inf}}), L, \sigma^2$
- 在实践中均未知

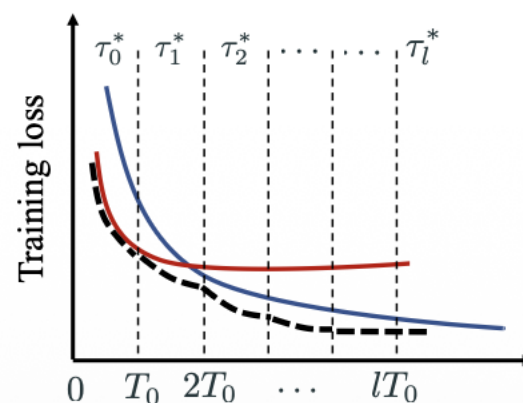
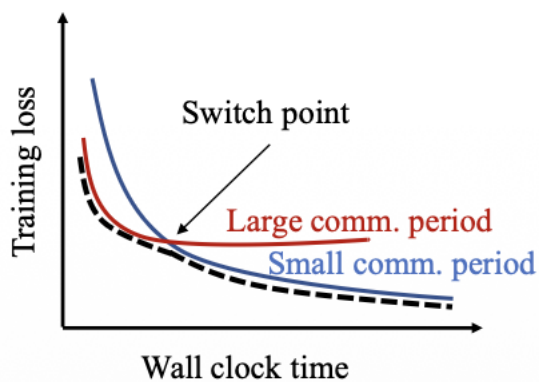
调整调度以适应实际约束

- 将时间划分为 T_0 大小的区间，并寻找每个区间的最佳 T
- 在第一个区间内，最优 T_0^* 为

$$\tau_0^* = \sqrt{\frac{2 [F(\mathbf{x}_{T=0}) - F_{\text{inf}}] D}{\eta^3 L^2 \sigma^2 T_0}}$$

- 通过假设工作节点从初始点开始计算 T_l

$$\tau_l^* = \sqrt{\frac{2 [F(\mathbf{x}_{T=lT_0}) - F_{\text{inf}}] D}{\eta^3 L^2 \sigma^2 T_0}}$$



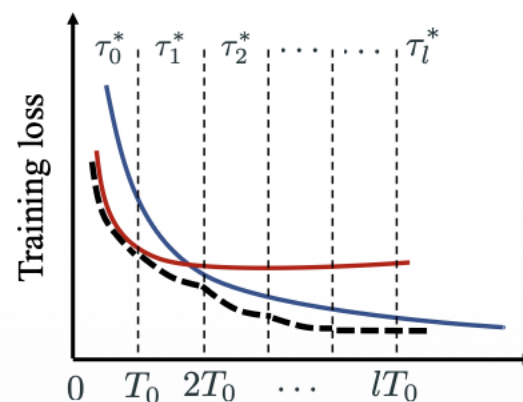
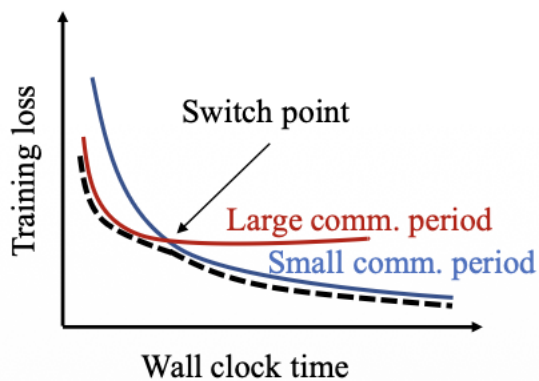
调整调度以适应实际约束

- 现在取比值于 τ_l^* 与 τ_0^* 得到

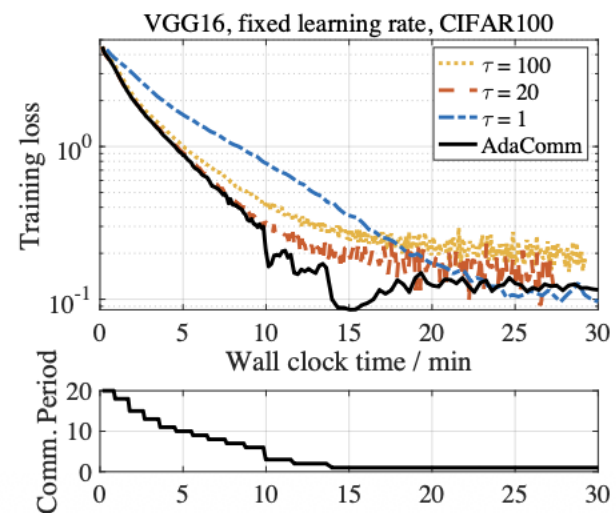
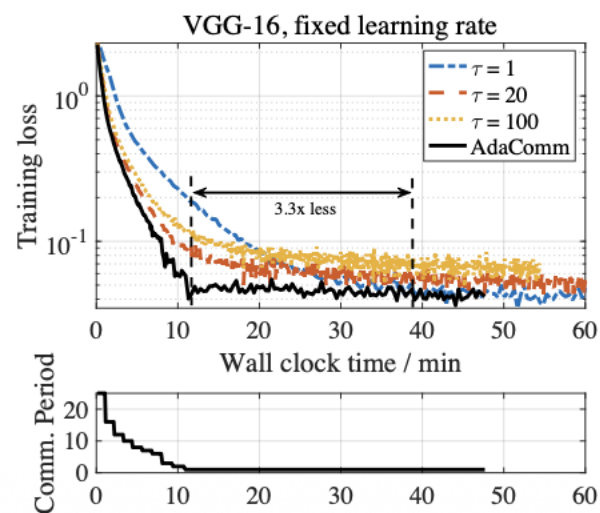
$$\tau_l^* = \sqrt{\frac{[F(\mathbf{x}_{T=lT_0}) - F_{\text{inf}}]}{[F(\mathbf{x}_{T=0}) - F_{\text{inf}}]}} \sim \sqrt{\frac{F(\mathbf{x}_{T=lT_0})}{F(\mathbf{x}_{T=0})}}$$

其中假设 $F_{\text{inf}}=0$

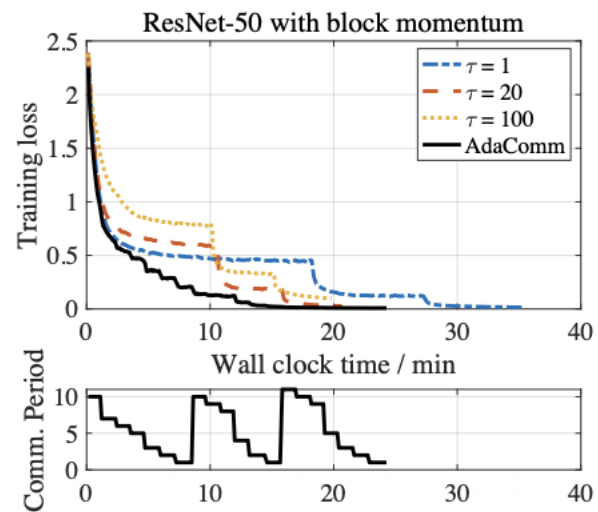
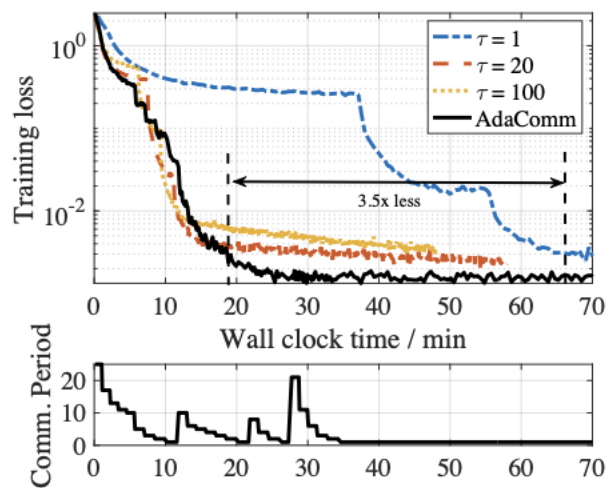
- 注意到未知项 L 和 σ^2 相互抵消
- 如何寻找 τ_0^* ? 执行网格搜索并尝试不同数值



AdaComm性能：固定学习率



AdaComm的性能：可变学习率



- 协作式SGD: <https://arxiv.org/pdf/1808.07576.pdf>
- 本地SGD: <https://arxiv.org/abs/1805.09767>
- Adacomm: <https://arxiv.org/pdf/1810.08313.pdf>

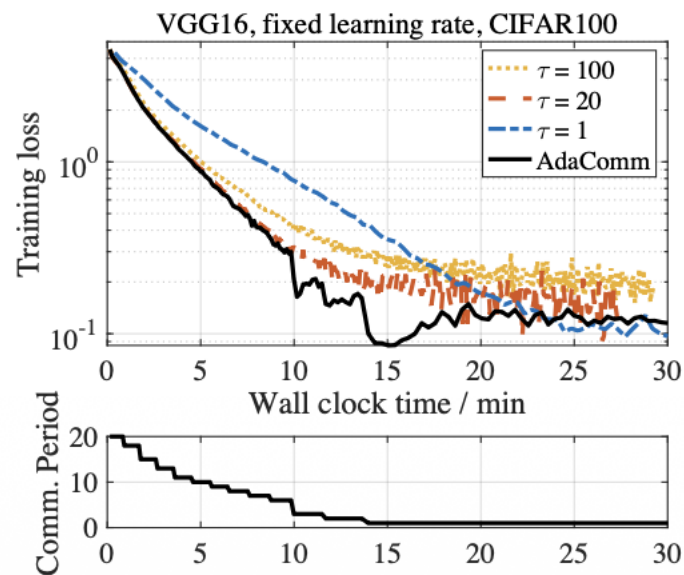
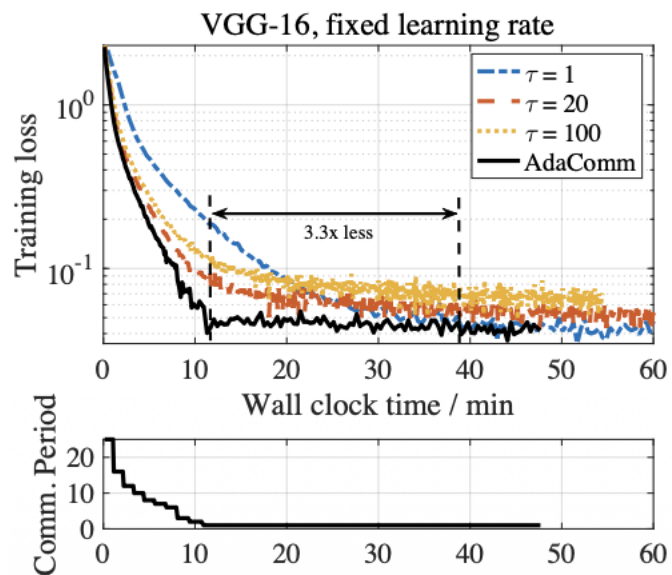
目录

Contents

- 1 本地更新随机梯度下降
- 2 **自适应通信方案**
- 3 弹性平均SGD
- 4 重叠本地SGD

AdaComm性能：固定学习率

- 实验分别训练VGG16在CIFAR10和CIFAR100数据集上
- Adacomm方案比固定 τ 方案显著更快



整合可变学习率

- 基础版AdaComm_T调度给定

I

$$\tau^* = \sqrt{\frac{2 [F(\mathbf{x}_1) - F_{\inf}] D}{\eta^3 L^2 \sigma^2 T}}$$

- 当取 τ_l^* 与 τ_0^* 的比值时, η 会被抵消。但如果已经在使用一种可变学习率调度呢?
- 解决方案:** 考虑在第 l 个大小为 T_0 的时间区间内, 学习率为 η_l 。现在取 τ_l^* 与 τ_0^* 的比值得到

$$\tau_l^* = \sqrt{\frac{\eta_0^3 [F(\mathbf{x}_{T=IT_0}) - F_{\inf}]}{\eta_l^3 [F(\mathbf{x}_{T=0}) - F_{\inf}]}} \tau_0^* \sim \sqrt{\frac{\eta_0^3 F(\mathbf{x}_{T=IT_0})}{\eta_l^3 F(\mathbf{x}_{T=0})}} \tau_0^*$$

整合可变学习率

- 针对给定学习率调度的自适应 τ 调度方案:

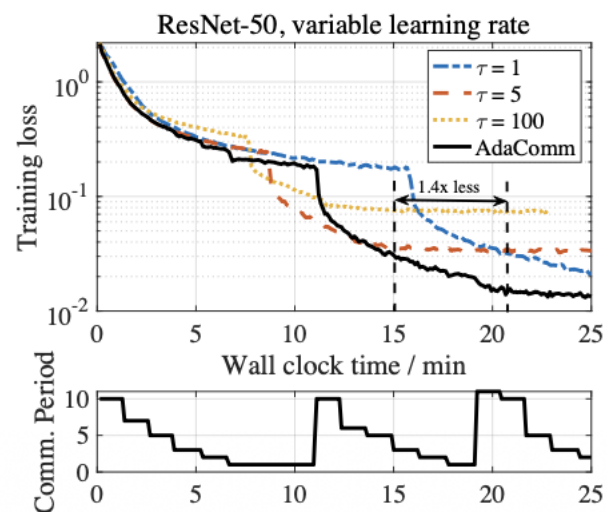
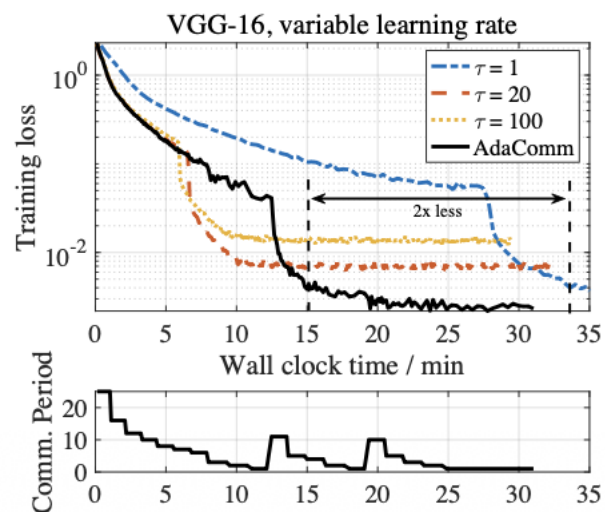
$$\tau_l^* = \sqrt{\frac{\eta_0^3}{\eta_l^3} \frac{[F(\mathbf{x}_{T=1/T_0}) - F_{\text{inf}}]}{[F(\mathbf{x}_{T=0}) - F_{\text{inf}}]}} \tau_0^* \sim \sqrt{\frac{\eta_0^3}{\eta_l^3} \frac{F(\mathbf{x}_{T=1/T_0})}{F(\mathbf{x}_{T=0})}} \tau_0^*$$

其中 τ_0^* 通过暴力网格搜索确定

- 问题:** 对于固定学习率, τ 如何随时间变化? 减少
- 问题:** 对于可变学习率, 若学习率在某一时刻衰减, 对应的 τ 会发生什么? 增加
- 这是因为, 当每一步更小时, 可以承受在不引起工作者间本地模型 $\mathbf{x}^{(i)}$ 与 $\mathbf{x}_k^{(i)}$ 差异的情况下, 采取更多局部步骤。

AdaComm的性能: 可变学习率

- 可变学习率
- AdaComm比固定 τ 方案收敛更快



在本地SGD设置中应用动量

- 回顾vanillaSGD中动量 γ 的加入

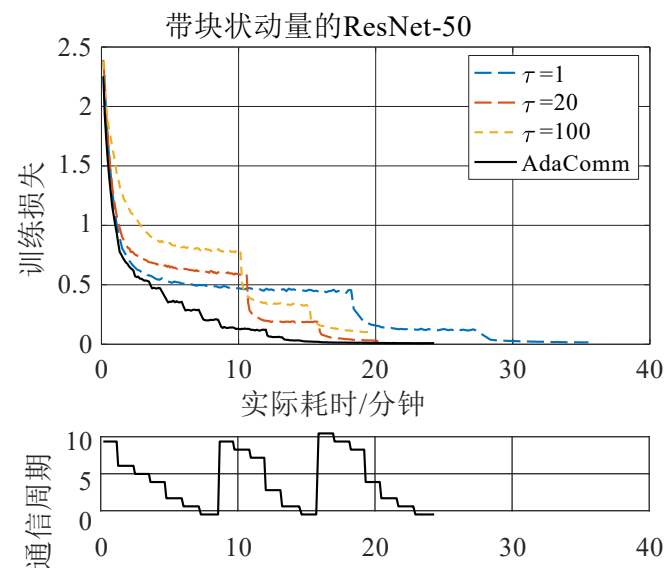
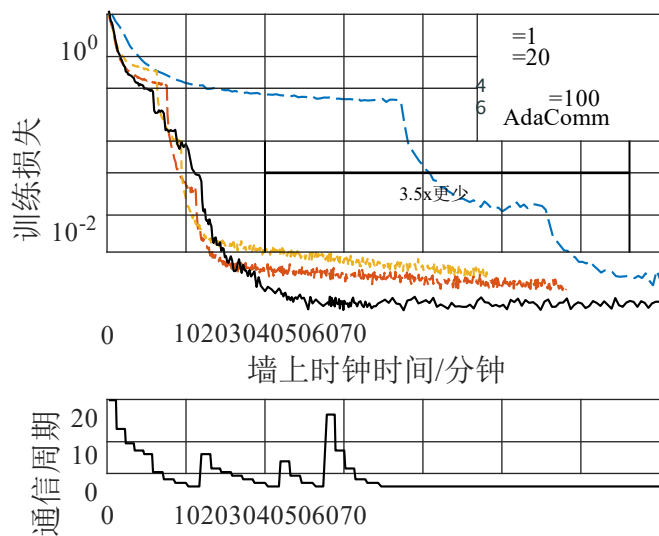
$$\mathbf{y}_t = \gamma \mathbf{y}_{t-1} + \eta \nabla F(\mathbf{x}_t)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{y}_t$$

- 在本地更新SGD中，可以使用动量进行本地更新，其中每个工作节点将维护其自己的动量缓冲区 $\mathbf{y}^{(i)}$ 。
- 但是每次模型平均时， $\mathbf{x}^{(i)}$ 都会发生剧烈变化；这可能导致动量出现波动，从而阻碍收敛
- 解决方案：**在每次平均步骤后，将工作节点上的**动量缓冲区 $\mathbf{y}$** 重置为零
- 除了本地动量，还可以在参数服务器上添加全局动量，通过将所有工作节点累计的本地更新视为单个梯度步骤

AdaComm的性能：块状动量效果显著

- 采用可变学习率与块状动量的结果
- AdaComm较固定 τ 方案收敛更快



目录

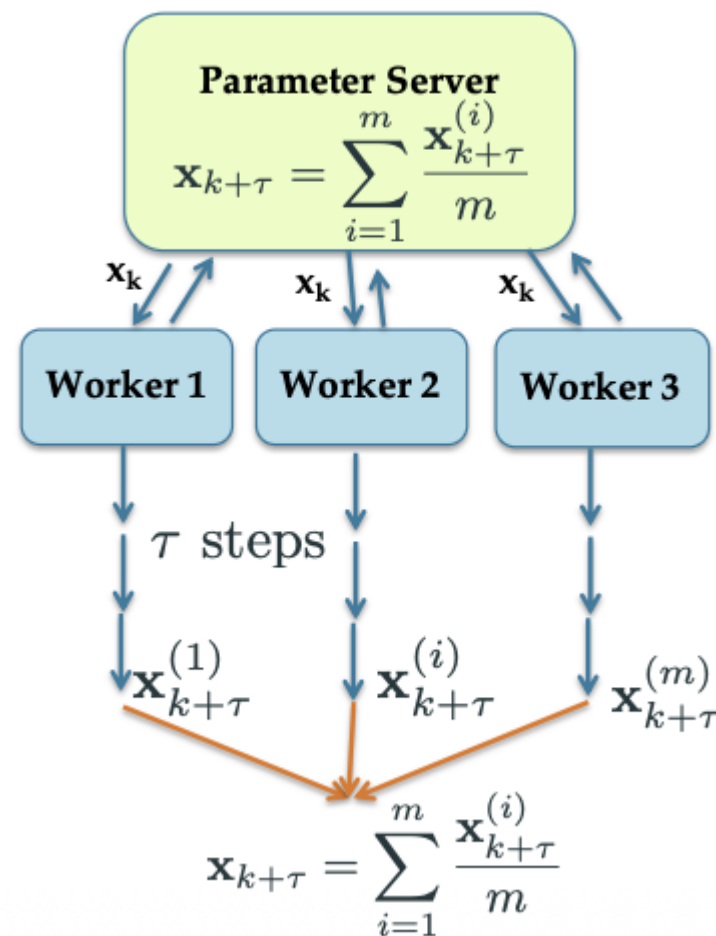
Contents

- 1 本地更新随机梯度下降
- 2 自适应通信方案
- 3 弹性平均SGD
- 4 重叠本地SGD

弹性平均SGD：直观理解

- 弹性平均允许工作节点模型之间有一定的松弛
- 在每次平均步骤中，参数服务器将工作节点模型拉向自己，而工作节点则共同将参数服务器模型拉向相反方向
- 为什么称为“弹性”？因为这种平均类似于用弹簧连接参数服务器与工作节点
- 弹性系数与 α 成正比。

弹性平均促进更多多样性在工作节点模型中



弹性平均随机梯度下降:目标函数

- 由张、克罗曼斯卡与扬·勒昆于2015年提出
- 在机器学习中通常解决的经验风险最小化问题

$$\min_{\mathbf{x}} F(\mathbf{x}) = \min_{\mathbf{x}} \sum_{i=1}^N f(\mathbf{x}; \xi_i)$$

- 希望以分布式方式进行训练, 其中 N 个数据样本被分配到 m 个工作节点上
- 参数服务器的模型 $\mathbf{x}$ 可能不同于工作节点模型的版本 $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$.
- 因此, 弹性平均随机梯度下降考虑一个改进的优化问题

$$\min_{\mathbf{x}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}} \sum_{i=1}^m \left[\underbrace{\sum_{n=1}^{N_i} f(\mathbf{x}^{(i)}; \xi_{i,n})}_{\text{Worker } i\text{'s local objective}} + \frac{\rho}{2} \|\mathbf{x}^{(i)} - \mathbf{x}\|^2 \right]$$

弹性平均随机梯度下降:算法

- 弹性平均随机梯度下降的目标函数:

$$\min_{\mathbf{x}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}} \sum_{i=1}^m \left[\sum_{n=1}^{N_i} f(\mathbf{x}^{(i)}; \xi_{i,n}) + \frac{\rho}{2} \|\mathbf{x}^{(i)} - \mathbf{x}\|^2 \right]$$

- 不同于同时更新所有变量 $\mathbf{x}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$:同一时间的做法,采用分步更新策略如下:

工作节点更新:
$$\mathbf{x}_{k+1}^{(i)} = \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)}) - \underbrace{\alpha(\mathbf{x}_k^{(i)} - \mathbf{x}_k)}_{\text{pulls } \mathbf{x}_k^{(i)} \text{ closer to } \mathbf{x}_k}$$

参数服务器更新:
$$\mathbf{x}_{k+1} = (1 - m\alpha)\mathbf{x}_k + m\alpha \underbrace{\left(\frac{1}{m} \sum_{i=1}^m \mathbf{x}_k^{(i)} \right)}_{\text{avg. of worker models}}$$

其中 $\alpha = \eta\rho$ 。

弹性平均随机梯度下降:算法

- 弹性平均更新规则:

工作节点更新: $\mathbf{x}_{k+1}^{(i)} = \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)}) - \underbrace{\alpha(\mathbf{x}_k^{(i)} - \mathbf{x}_k)}_{\text{pulls } \mathbf{x}_k^{(i)} \text{ closer to } \mathbf{x}_k}$

参数服务器更新: $\mathbf{x}_{k+1} = (1 - m\alpha)\mathbf{x}_k + m\alpha \underbrace{\left(\frac{1}{m} \sum_{i=1}^m \mathbf{x}_k^{(i)} \right)}_{\text{avg. of worker models}}$

其中 $\alpha = \eta\rho$ 。

- 对称强制 $\alpha(\mathbf{x}_k^{(i)} - \mathbf{x}_k)$: 于每个 $\mathbf{x}$ 与 $\mathbf{x}$ $\mathbf{x}_k^{(i)}$ $\mathbf{x}_k$
- 较小 α : 允许更多松弛在工作节点与参数服务器模型间, 鼓励更多局部探索
- 较大 α : 紧密耦合工作节点与参数服务器模型, 较少探索

弹性平均本地更新随机梯度下降

- 基础弹性平均更新规则:

工作节点更新:
$$\mathbf{x}_{k+1}^{(i)} = \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)}) - \underbrace{\alpha(\mathbf{x}_k^{(i)} - \mathbf{x}_k)}_{\text{pulls } \mathbf{x}_k^{(i)} \text{ closer to } \mathbf{x}_k}$$

参数服务器更新:
$$\mathbf{x}_{k+1} = (1 - m\alpha)\mathbf{x}_k + m\alpha \underbrace{\left(\frac{1}{m} \sum_{i=1}^m \mathbf{x}_k^{(i)} \right)}_{\text{avg. of worker models}}$$

假设平均化在每次迭代 k 后进行

- 相反, 为减少通信延迟, 可允许工作节点在每次平均化前进行 T 次本地更新其所得模型。

弹性平均本地更新随机梯度下降

- 弹性平均本地更新随机梯度下降 (始于 $\mathbf{x}_0$)

工作节点更新: $\mathbf{x}_{k+\frac{1}{2}}^{(i)} = \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)})$

$$\mathbf{x}_{k+1}^{(i)} = \begin{cases} \mathbf{x}_{k+\frac{1}{2}}^{(i)} - \alpha(\mathbf{x}_k^{(i)} - \mathbf{x}_k) & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_{k+\frac{1}{2}}^{(i)} & \text{otherwise} \end{cases}$$

参数服务器更新:

$$\mathbf{x}_{k+1} = \begin{cases} (1 - m\alpha)\mathbf{x}_k + m\alpha \left(\frac{1}{m} \sum_{i=1}^m \mathbf{x}_k^{(i)} \right) & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_k & \text{otherwise} \end{cases}$$

- 通过通信频率降低一个因子 τ

弹性平均SGD的误差收敛

- 原始EASGD论文对二次目标函数进行了分析
- 协作式SGD提供了对一般非凸目标函数的分析

弹性平均SGD的收敛性

对于一个L-光滑函数，且初始点为 $\mathbf{x}_1$ ，那么经过t次弹性平均SGD迭代后， $F(\mathbf{x}_t)$ 满足如下界：

$$\mathbb{E} \left[\frac{1}{t} \sum_{k=1}^t \|\nabla F(\mathbf{x}_k)\|^2 \right] \leq \frac{2(m+1)[F(\mathbf{x}_1) - F_{\inf}]}{\eta m t} + \frac{\eta L \sigma^2}{m+1} + \eta^2 L^2 \sigma^2 \left(\frac{1 + \zeta^2}{1 - \zeta^2} \tau - 1 \right)$$

其中， $\mathbf{x}_k$ 表示第k次迭代时的平均模型，并且 $\zeta = \max(|1 - \alpha|, |1 - (m+1)\alpha|)$.

- 适当增加 α （但不过度）会使 ζ 减小，从而降低误差下限

α 的最优值

- 当 $\alpha=2/(m+2)$ 时, 收敛时的误差下限达到最小
- 对于此 α , 当 $r=1$ 时的收敛界如下

弹性平均SGD的收敛性

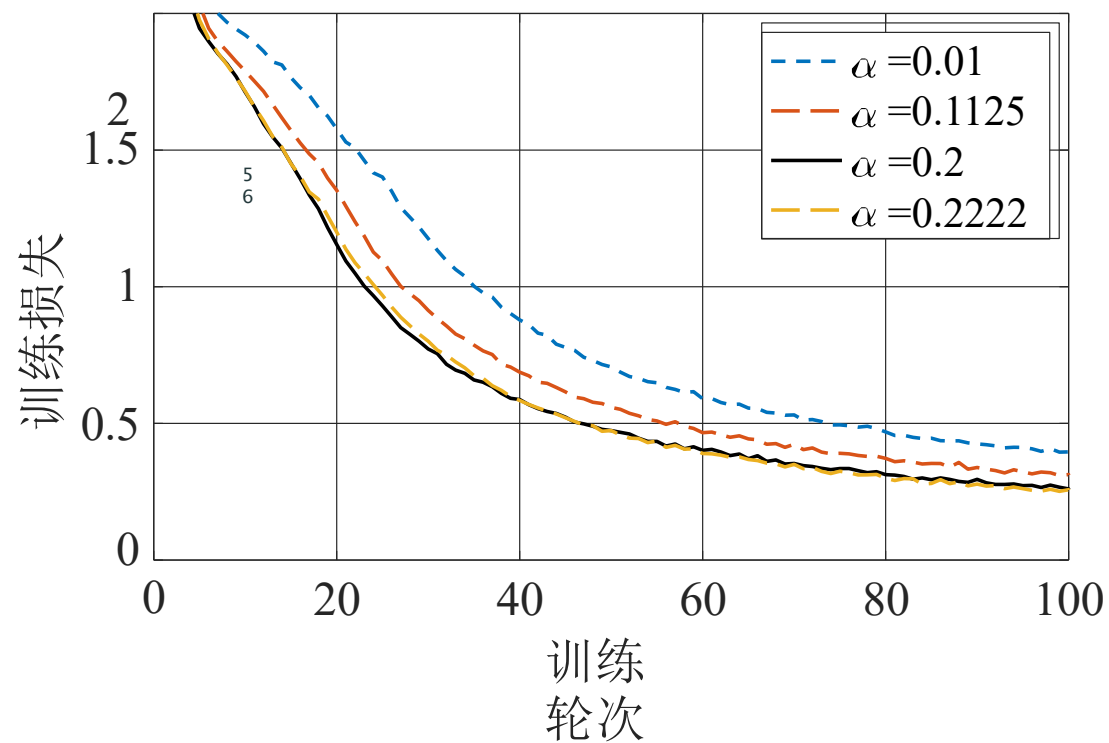
对于一个 L -光滑函数, 且初始点为 $\mathbf{x}_1$, 那么经过 t 次弹性平均SGD迭代后, $F(\mathbf{x}_t)$ 满足如下界:

$$\mathbb{E} \left[\frac{1}{t} \sum_{k=1}^t |\nabla F(\mathbf{x}_k)|^2 \right] \leq \frac{2(m+1)[F(\mathbf{x}_1) - F_{\inf}]}{\eta m t} + \frac{\eta L \sigma^2}{m+1} + \frac{1}{2} \eta^2 L^2 \sigma^2 \left(\frac{m^2}{m+1} \right)$$

其中, $\mathbf{x}_k$ 表示第 k 次迭代时的平均模型。

参数 α 对收敛性的影响

- 8个工作节点, 因此最优 α 值为 $2/(8+2)=0.2$
- 本地更新次数 $T=1$



弹性平均SGD每次迭代的运行时间

回顾弹性平均本地更新SGD算法:

工作者更新步骤: $\mathbf{x}_{k+\frac{1}{2}}^{(i)} = \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)})$

$$\mathbf{x}_{k+1}^{(i)} = \begin{cases} \mathbf{x}_{k+\frac{1}{2}}^{(i)} - \alpha(\mathbf{x}_k^{(i)} - \mathbf{x}_k) & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_{k+\frac{1}{2}}^{(i)} & \text{otherwise} \end{cases}$$

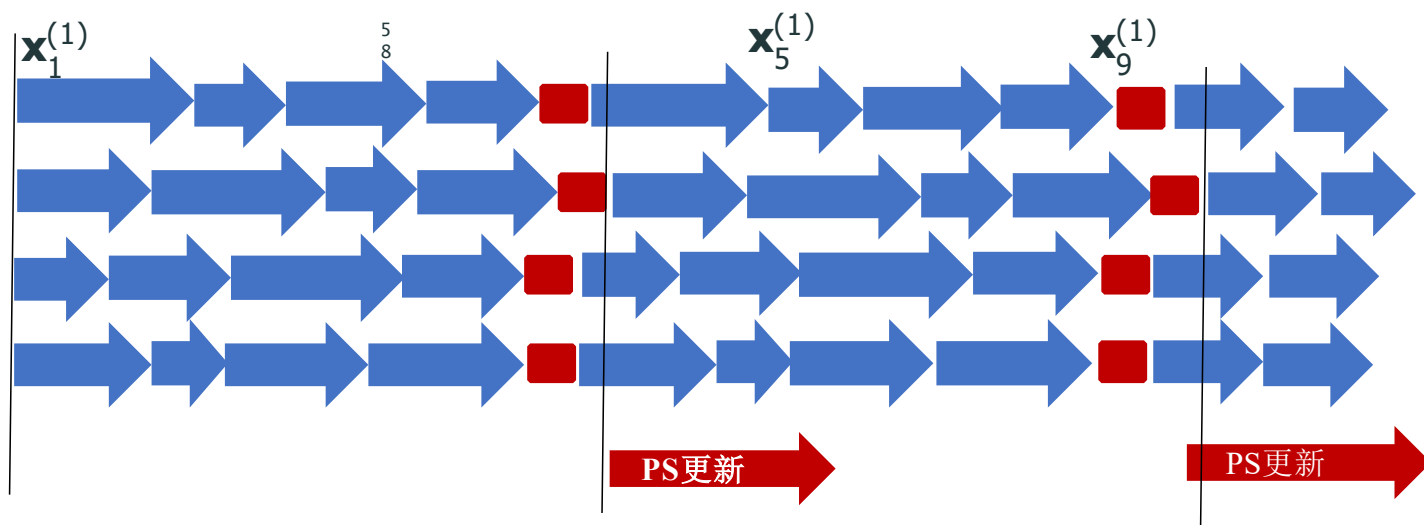
参数服务器更新:

$$\mathbf{x}_{k+1} = \begin{cases} (1 - m\alpha)\mathbf{x}_k + m\alpha \left(\frac{1}{m} \sum_{i=1}^m \mathbf{x}_k^{(i)} \right) & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_k & \text{otherwise} \end{cases}$$

- 由于 $\mathbf{x}_{aT} = \mathbf{x}_{aT+1} = \mathbf{x}_{aT+\tau-1}$, 因此工作节点每隔 τ 次迭代仅需访问到参数 $\mathbf{x}_k$
- 因此, 参数服务器PS的更新可以与工作节点的更新并行进行, 而非等待其完成后再执行

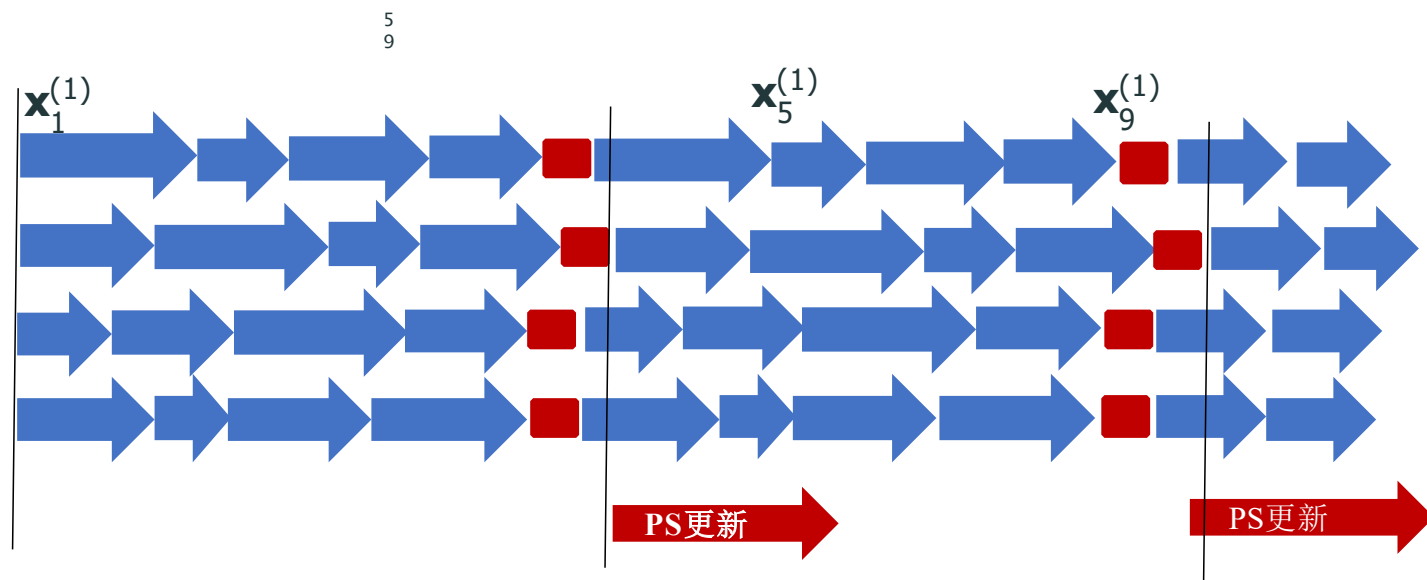
弹性平均SGD每次迭代的运行时间

- 蓝色箭头表示工作节点的本地更新过程
- 红色箭头表示参数服务器PS的更新
- 小型红色方块表示工作节点将更新后的模型发送至参数服务器PS所需的通信时间



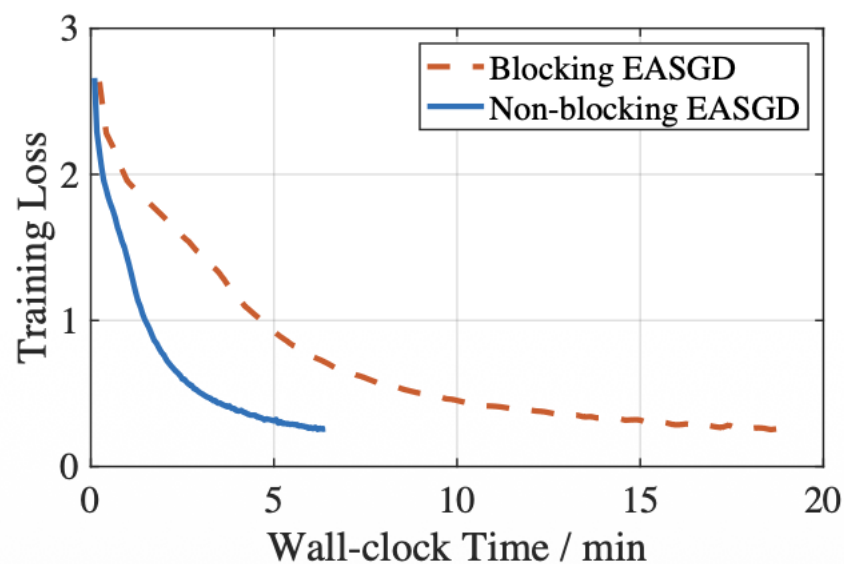
阻塞式与非阻塞式实现

- 在非阻塞式实现中，参数服务器PS的更新会与工作节点的更新并行进行（如下文所示）
- 在阻塞式实现中，强制参数服务器PS的更新必须在工作节点更新完成后执行。因此，每一组蓝色箭头（工作节点更新）之后都会跟随一个红色箭头（PS更新），接着是新一轮工作节点的更新——整个流程按顺序执行



阻塞式与非阻塞式实现

- 在非阻塞式实现中，参数服务器PS的更新会与工作节点的更新并行进行
- 在阻塞式实现中，强制参数服务器PS的更新必须在工作节点更新完成后执行。因此，每一组蓝色箭头（工作节点更新）之后都会跟随一个红色箭头（PS更新），接着是新一轮工作节点的更新——整个流程按顺序执行



目录

Contents

- 1 本地更新随机梯度下降
- 2 自适应通信方案
- 3 弹性平均SGD
- 4 重叠本地SGD**

采用重叠式本地随机梯度下降的动机

回顾弹性平均随机梯度下降的更新规则：

工作节点更新: $\mathbf{x}_{k+\frac{1}{2}}^{(i)} = \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)})$

$$\mathbf{x}_{k+1}^{(i)} = \begin{cases} \mathbf{x}_{k+\frac{1}{2}}^{(i)} - \alpha(\mathbf{x}_k^{(i)} - \mathbf{x}_k) & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_{k+\frac{1}{2}}^{(i)} & \text{otherwise} \end{cases}$$

参数服务器更新:

$$\mathbf{x}_{k+1} = \begin{cases} (1 - m\alpha)\mathbf{x}_k + m\alpha \left(\frac{1}{m} \sum_{i=1}^m \mathbf{x}_k^{(i)} \right) & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_k & \text{otherwise} \end{cases}$$

- 在参数服务器模型中EASGD具有惯性——其更新是先前参数服务器模型与工作节点回传模型的加权组合
- 这可能减缓其收敛速度

采用重叠式本地随机梯度下降的动机

重叠式本地随机梯度下降对此进行如下改进：

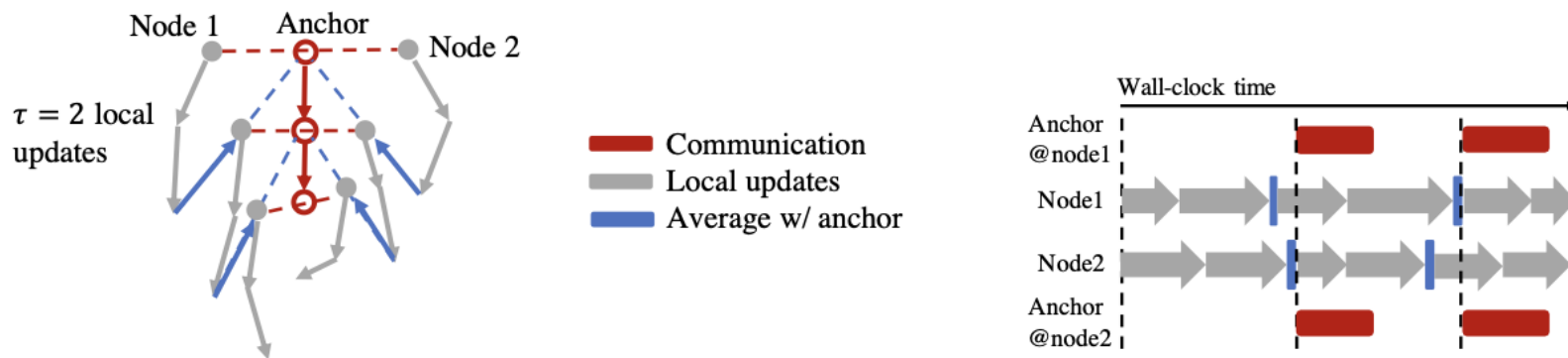
工作节点更新：

$$\mathbf{x}_{k+\frac{1}{2}}^{(i)} = \mathbf{x}_k^{(i)} - \eta g(\mathbf{x}_k^{(i)})$$
$$\mathbf{x}_{k+1}^{(i)} = \begin{cases} \mathbf{x}_{k+\frac{1}{2}}^{(i)} - \alpha(\mathbf{x}_{k+\frac{1}{2}}^{(i)} - \mathbf{x}_k) & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_{k+\frac{1}{2}}^{(i)} & \text{otherwise} \end{cases}$$

参数服务器更新：

$$\mathbf{x}_{k+1} = \begin{cases} \frac{1}{m} \sum_{i=1}^m \mathbf{x}_{k+1}^{(i)} & \text{if } (k+1) \bmod \tau = 0 \\ \mathbf{x}_k & \text{otherwise} \end{cases}$$

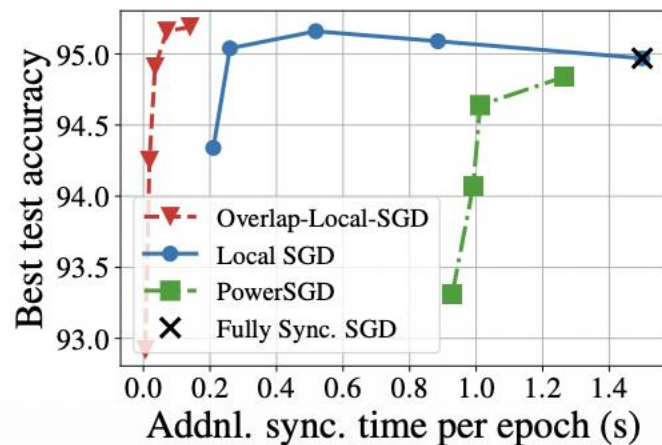
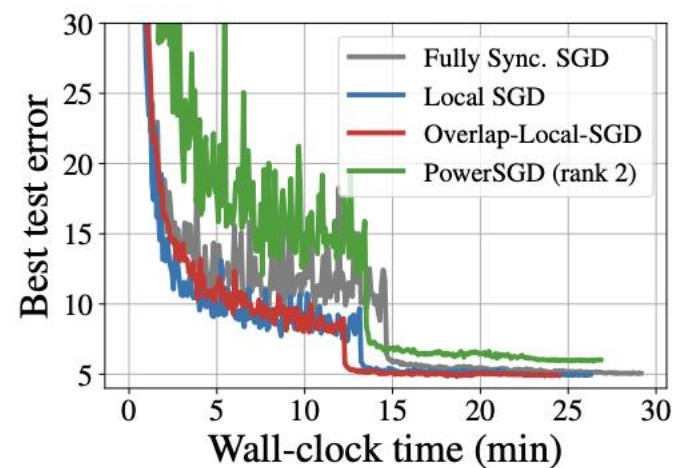
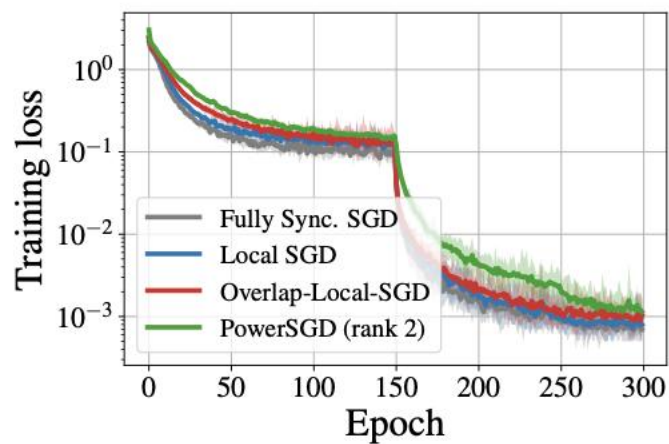
重叠式本地随机梯度下降示意图



- 锚点指代存储在节点的参数服务器模型副本
- 弹性平均随机梯度下降的轻度改良版
- 保持了采用非阻塞式实现的灵活性

重叠本地随机梯度下降的性能

在CIFAR10上训练ResNet-18的结果



- Adacomm:<https://arxiv.org/pdf/1810.08313.pdf>
- 协作式SGD:<https://arxiv.org/pdf/1808.07576.pdf>
- 弹性平均SGD:<https://arxiv.org/pdf/1412.6651.pdf>
- 重叠本地SGD:<https://arxiv.org/pdf/2002.09539.pdf>